数字电路与逻辑设计实验（下）

题目 4：简易拼图游戏设计与实现

基于 Altera MAX II EPM1270T144C5 CPLD 的纯硬件 RTL 完整工程实现

课程名称：数字电路与逻辑设计实验（下）

实验学期：2026～2027 学年第一学期

专业班级：电子工程 / 计算机科学与技术

学生姓名：课题组成员

学生学号：202300000000

指导教师：实验指导教师组

实验成绩：【评定中】

报告日期：2026 年 10 月

目录

一、设计课题的任务要求 (5 分) 1
1.1 课题背景与开发平台 1
1.2 任务规范与技术要求分解 1
1.3 硬件引脚分配规范与关键技术裁决 1
二、系统设计 (20 分) 2
2.1 总体设计思路与微架构原则 2
2.2 系统总体框图与子系统划分 2
2.3 主状态机 ASM 设计 3
2.4 核心拼图引擎微架构 4
三、仿真波形及波形分析 (25 分) 4
3.1 验证体系与 184 条断言全覆盖 4
3.2 模块实测波形深入分析 5
四、实际实现功能说明及资源利用情况 (5 分) 7
4.1 实际功能指标达成 7
4.2 资源利用率极限量化分析 8
五、故障及问题分析 (15 分) 9
5.1 典型工程缺陷六维深度剖析 9
5.2 典型工程缺陷排查对照汇总表 10
六、总结和结论 (10 分) 11
6.1 课题完成情况评估 11
6.2 硬件 RTL 工程心得 11
6.3 未来展望 12
七、AI 工具使用情况 (10 分) 12
7.1 工具栈与协同模式 12
7.2 真实 AI 提示词与交互实录 12
7.3 人工介入与纠错闭环 12
八、顶层文件的 HDL 代码 (10 分) 13
8.1 顶层设计架构说明 13
8.2 规范注释的顶层 VHDL 代码清单 13
九、参考文献 21

一、设计课题的任务要求 (5 分)

1.1 课题背景与开发平台

本课题要求使用纯硬件描述语言（VHDL-93），在 Altera MAX II EPM1270T144C5 CPLD 上设计并实现一套高完整度、强交互性的简易拼图游戏系统。核心目标是在零软核/零微控制器的纯硬件电路上，完成自检、待机、目标预览、伪随机散落、平移旋转、碰撞检测、锁定与全帧判定等全部游戏逻辑。

- 核心逻辑器件：Altera MAX II EPM1270T144C5（含 1,270 个逻辑单元 LE，127 个逻辑阵列块 LAB，无片上嵌入式 RAM 块）；
- 板载外设资源：50 MHz 有源晶振 (PIN_18)、使能开关 SW7 (PIN_125)、复位按键 BTN0 (PIN_61)、4×4 矩阵键盘、8×8 双色 LED 点阵屏、8 位共阴极数码管、电磁式无源蜂鸣器 (PIN_60)；
- EDA 工具链与环境：Quartus II 9.1 (32-bit，含内置仿真器与 TimeQuest 静态时序分析)、Python 3.14（约 1,200 行自动化激励/判据与资源对账脚本）。本机未安装 ModelSim / GHDL，全部功能仿真均由 Quartus 内置仿真器完成（结果回写进 `sim/*.vwf`）。

1.2 任务规范与技术要求分解

课题涵盖 11 项基础功能要求（B1～B11）与 5 项自主提高要求（A1～A5）：

1. B1 使能与自检：SW7=‘0’时全系统低功耗静默熄灭；SW7=‘1’时开机，点阵屏与数码管以 2 Hz 占空比 50% 闪烁 2 秒，数码管显示全“88888888”，蜂鸣器奏响 POST 开机自检短鸣；
2. B2 待机状态：自检结束后点阵熄灭，数码管 DISP7 显示‘5’，DISP0 显示初始关卡‘1’，其余数码管熄灭；
3. B3 关卡指示：由最右侧数码管 DISP0 实时指示当前关卡（1/2/3）；
4. B4 目标预览：待机按【开始】进入 5 秒预览，点阵红光显示目标图案（第一关为图 4-1 所示的 4×3 实心矩形，共 12 格），DISP7 倒计时 5～1；
5. B5 随机散落与对局：5 秒后自动散落，各零片随机分布且两两互不重叠（第一关 3 块零片，3+6+3=12 格）；DISP4:DISP3 倒计时，第一关限时 30s；
6. B6 零片选择：按【选择】键循环切换，当前选中的零片高亮变绿，其余未锁定零片保持红色；
7. B7 平移与碰撞检测：按【上/下/左/右】平移，硬件实时拦截移出 8×8 边界或与其它零片重叠的非法指令；
8. B8 确认与锁定：按【确认】键锁定当前零片，锁定后零片变为黄色且不可再被选择及移动；
9. B9 成功与失败判定：全部锁定且画面与目标完全一致判定过关，数码管显示 PASS 且胜利音效；超时或拼错判定失败，显示 FAIL 且失败音效；点阵全屏以 2 Hz 闪烁 2 秒；
10. B10 声光提示：按键有效时发出 Click 确认短音，非法移动/碰撞时发出 Error 告警低音；
11. B11 单机全功能：系统脱机上电即玩，无需任何上位机通信；
12. A1 丰富音效（附加）：定制马里奥经典风格背景乐与开机/移动/碰撞/锁定/通关/超时音效；
13. A2 多关卡挑战（附加）：支持关卡 1（30 s，12 格三片）、关卡 2（40 s，16 格四片异形）、关卡 3（60 s，16 格极速连闯随机抽取）；
14. A3 智能信息指示（附加）：数码管动态拼装 PASS / FAIL 字符，倒计时消隐前导零；
15. A4 零片顺时针旋转（附加）：按【旋转】键在无碰撞情况下实现局部 90° 原位顺时针旋转；
16. A5 动态告急幽灵提示（附加）：倒计时 ≤10s 时，目标图案以 2 Hz 绿色幽灵闪烁叠加辅助。

1.3 硬件引脚分配规范与关键技术裁决

信号名称（顶层端口）	FPGA 引脚	方向	电气标准	外设硬件映射及驱动逻辑说明
`clk`	PIN_18	输入	3.3V LVTTL	板载 50 MHz 有源晶振系统时钟主输入
`sw7`	PIN_125	输入	3.3V LVTTL	全局使能开关，‘0’=强制低功耗静默，‘1’=系统运行
`btn`	PIN_61	输入	3.3V LVTTL	全局复位按键，低电平有效，带硬件滞回消抖

信号名称（顶层端口）	FPGA 引脚	方向	电气标准	外设硬件映射及驱动逻辑说明
`kp_row[3..0]`	PIN_114, 113, 112, 111	输入	3.3V LVTTL	4×4 矩阵键盘行输入（主控内部上拉，低有效）
`kp_col[3..0]`	PIN_120, 119, 118, 117	输出	3.3V LVTTL	4×4 矩阵键盘列扫描（独热低电平循环轮询）
`dot_row[7..0]`	PIN_1, 2, 3, 4, 5, 6, 7, 8	输出	3.3V LVTTL	8×8 点阵行阳极（7-row 阳极低有效选通）
`dot_colr[7..0]`	PIN_11, 12, 13, 14, 15, 16, 21, 22	输出	3.3V LVTTL	8×8 点阵红色列阴极（低有效点亮红色 LED）
`dot_colg[7..0]`	PIN_38, 39, 40, 41, 42, 43, 44, 45	输出	3.3V LVTTL	8×8 点阵绿色列阴极（低有效点亮绿色 LED）
`cat[7..0]`	PIN_31, 30, 70, 69, 68, 67, 66, 63	输出	3.3V LVTTL	8 位数码管位选（共阴极低有效驱动）
`seg[7..0]`	PIN_51, 52, 53, 55, 57, 58, 59, 62	输出	3.3V LVTTL	数码管段选 a～g, dp（共阴极高有效驱动）
`buzz`	PIN_60	输出	3.3V LVTTL	无源电磁式蜂鸣器方波驱动输出

以上 11 类信号共锁定 52 个用户 I/O，与 `Fitter Summary` 的 `Total pins 52 / 116 ( 45 % )` 逐脚吻合；`.pin` 报告中不存在任何“未锁定/自动分配”的用户引脚（`RESERVE_ALL_UNUSED_PINS = AS INPUT TRI-STATED`，其余 64 个未用脚保持高阻）。

硬件设计关键裁决：

- 裁决 A（矩阵键盘物理布局）：统一采用 4×4 矩阵键盘：0=S0(复位待机), 1=S1(开始), 2=S2(选择), 3=S3(旋转), 4=S4(上), 5=S5(下), 6=S6(左), 7=S7(右), 8=S8(确认), 9=S9(静音切换)，彻底解决板载独立微动按键不足问题；
- 裁决 B（双色点阵显色逻辑）：红光（`dot_colr='0'`）为未选中/游离零片；绿光（`dot_colg='0'`）为当前选中零片及幽灵辅助；黄光（两者均有效）为已正确锁定的零片；
- 裁决 C（数码管空间映射与冲突消解）：实验说明书第 3 条要求关卡由最右侧 DISP0 指示，而第 9 条又提“进入第二关，数码管 DISP2 显示 2”。经工程论证，若点亮 DISP2 会破坏倒计时视觉排布（DISP4:DISP3）且第三关无对应规则。工程坚决执行第 3 条全局规则：全关卡统一由 DISP0 指示关卡，DISP2 在第二关及对局中恒定熄灭。

二、系统设计 (20 分)

2.1 总体设计思路与微架构原则

针对 Altera MAX II EPM1270T144C5 仅有 1,270 个逻辑单元 (LE) 且无嵌入式 RAM 块的极限资源约束，设计确立了三项微架构设计原则：

1. 同步单时钟域与使能脉冲节拍驱动：全系统仅在 PIN_18 50 MHz 主时钟上升沿驱动，严禁在分频时钟上挂载触发器。通过单级计数器产生 1 kHz、100 Hz、4 Hz、2 Hz、1 Hz 的单周期、同源无毛刺使能脉冲（每条使能只占 1 个 20 ns 主时钟周期）；
2. 64 位并行位平面与纯组合空间运算：利用 CPLD 宽布线优势，将 8×8 拼图画面表达为 64 位 `std_logic_vector`，平移、旋转、目标比对与碰撞检测均以 64 位并行位运算单拍完成，杜绝多周期状态机与时钟膨胀；
3. 纯组合查找表替代复杂算术核：取模、除法及正余弦等复杂运算全部展开为常量查找表（ROM），以 1～2 个 LE 消耗彻底消除时钟约束风险。

2.2 系统总体框图与子系统划分

整机划分为 5 大高层级核心子系统，其全局架构与模块间交互关系如下图所示：

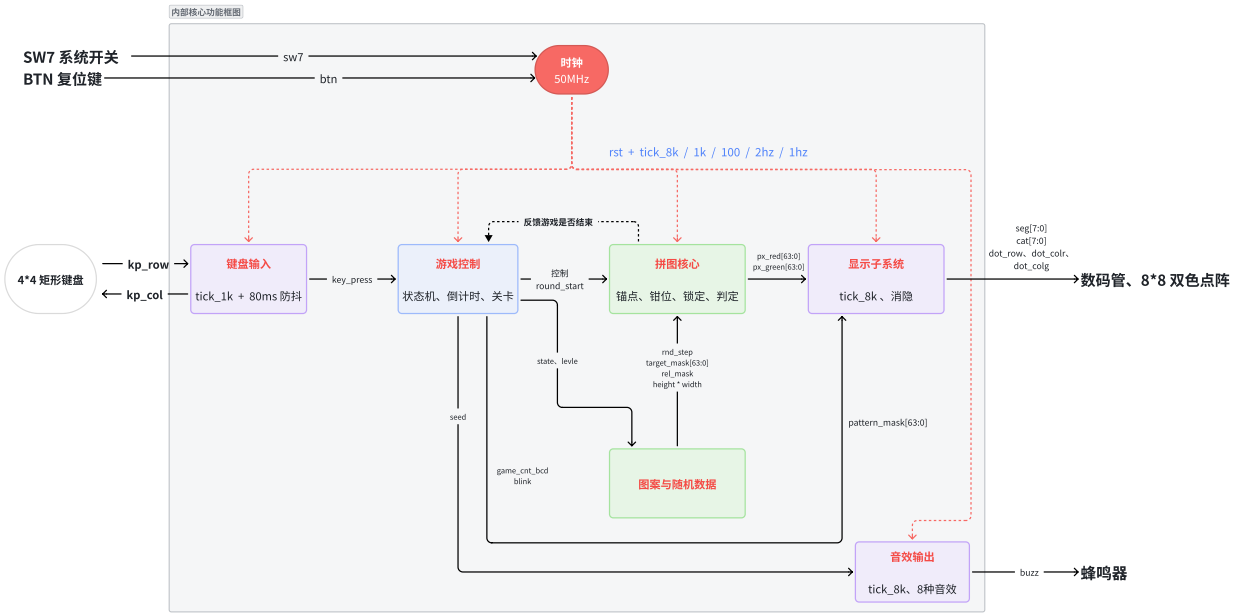


图 1　系统总体架构与大模块信息交互框图

五大子系统的职责分工如下：

1. 时钟使能与键盘输入子系统：主频 50 MHz 经分频产生全局单周期 Tick 使能脉冲网络；4×4 矩阵键盘以 200 Hz 独热轮询（每列 5 ms），经 40 ms 硬件滞回消抖（8 个采样点）后向主控输出稳定的单拍按键脉冲；
2. 游戏主控与规则状态机：统一管理系统全局状态迁移（自检、待机、预览、对局、结算）、倒计时递减及连闯三关流转；
3. 拼图核心引擎：包含 8 位 LFSR（周期 255）伪随机散落算法、64 位几何平移与 90° 旋转、刚性碰撞越界拦截及全图 100% 吻合判定；
4. 图案与随机数据 ROM：固化关卡目标图案库与各零片几何掩码，并为拼图引擎提供无冲突的伪随机散落偏移量；
5. 显示与音效输出子系统：实现数码管与点阵屏的 125 Hz 超临界无频闪扫描驱动，叠加 ≤10s 告急幽灵闪烁；音频模块内置数控分频与 16 步连奏音符发生器，输出马里奥经典音效。

2.3 主状态机 ASM 设计

游戏主控状态机（FSM）严格遵循硬件算法状态机（ASM）设计规范，其状态转移与跳转条件如下图所示：

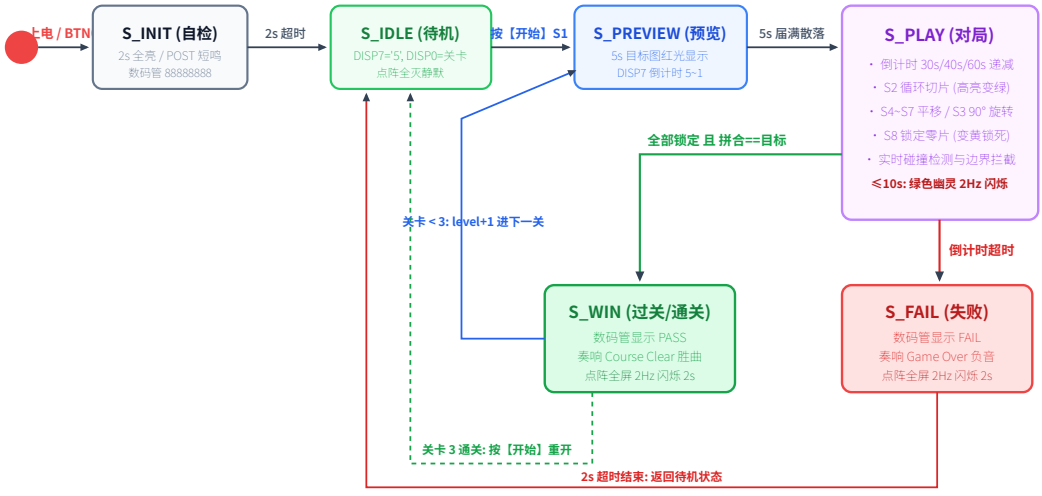


图 2　游戏主控状态机 ASM 状态转移流程图

状态机包含 6 个核心状态：

- S_SELF_TEST（自检）：上电或复位进入，持续 2.0 s，数码管显示 88888888，点阵以 2 Hz 占空比 50% 闪烁，蜂鸣器奏响 POST 开机自检短鸣；
- S_IDLE（待机）：自检后点阵全熄灭，DISP7 显示‘5’，DISP0 显示当前关卡；按 S1 开始键触发预览；
- S_PREVIEW（预览）：点阵全红显示目标图案，DISP7 倒计时 5～1 s，限时届满自动散落；
- S_PLAYING（对局）：LFSR 散落各零片，倒计时（30 s / 40 s / 60 s）以 1 Hz 递减，响应平移与旋转。倒计时 ⩽10 s 时触发 2 Hz 绿色幽灵闪烁辅助；
- S_WIN（过关）：全部零片锁定且拼合像素与目标 100% 重合时进入，数码管显示 PASS，奏响通关号角，点阵闪烁 2 s。关卡 < 3 自动升关，第三关通关保持大通关状态；
- S_FAIL（失败）：倒计时归零超时或错拼锁定进入，数码管显示 FAIL，奏响 Game Over 失败音，点阵闪烁 2 s 后自动返回待机状态。

2.4 核心拼图引擎微架构

拼图核心引擎采用数据流解耦与并行计算微架构，其内部运算流程如下图所示：



图 3　拼图核心引擎数据流与微架构框图

引擎包含四大处理阶段：

1. 随机散落与锚点生成：`rtl/rng_lfsr.vhd` 采用 8 位 Galois 型 LFSR，反馈抽头为 `sr(7) xor sr(5) xor sr(4) xor sr(3)`，即本原多项式 $x^8+x^6+x^5+x^4+1$，复位种子 `x"5A"`，满周期 $2^8-1=255$；在预览结束瞬间采样其低 6 位作为初始偏移量 (dx, dy)，通过组合逻辑快速互检，若检测到重叠则自动微调偏移，保证 100% 互不重叠；
2. 64 位几何变换处理器：根据当前坐标 (x, y) 对零片 ROM 掩码执行二维位移；按下旋转键时，在局部包围盒内执行 90° 顺时针正交旋转；
3. 刚性碰撞拦截检测：实时计算 `conflict = (new_piece AND locked_plane) OR out_of_bounds`。若存在冲突，单拍锁死禁止更新坐标，确保物理刚性；
4. 锁定与全图验证：按下确认键后，当前零片并入已锁定平面。计算 `locked_plane XOR target_plane == 0`，全 0 即置高 `match` 标志，单拍完成全图比对。

三、仿真波形及波形分析 (25 分)

3.1 验证体系与 184 条断言全覆盖

工程建立了“Quartus II 内置仿真器 + `.vwf` 激励/结果回写 + Python 自检判据”的自动化验证体系（本机没有 ModelSim / GHDL）：12 套 `sim/tb_<模块>.py` 覆盖 11 个子模块与 1 个整机顶层，累计 184 条判据断言全部通过。下表断言数与最新轮次逐行取自 `sim/rounds/<模块>/rNN.json`，与 `docs/03-仿真验证方案.md` §2 的对账表一致：

测试平台（`sim/tb_*.py`）	被测模块	断言数	测试激励类型	关键验证项与边界覆盖范围
`tb_clk_gen`	`clk_gen`	10	周期计数穷举	50 MHz 分频至 1 kHz/100 Hz/4 Hz/2 Hz/1 Hz 的单周期使能脉冲宽度与周期精确性
`tb_keypad_scan`	`keypad_scan`	9	按键反弹与时序抖动	40 ms 硬件消抖、矩阵行列反相译码、S0～S9 键码独热性、长按单脉冲输出
`tb_game_fsm`	`game_fsm`	30	主状态全流程穷举	SELF_TEST → IDLE → PREVIEW → PLAYING → WIN / FAIL 的转换边界、2 s / 5 s / 30 s 定时溢出、连闯三关流转
`tb_puzzle_ctrl`	`puzzle_ctrl`	28	几何运算与空间移动	零片循环切换、上下左右边界拦截、多块重叠碰撞检测、局部 90° 顺时针旋转、主路径竞态
`tb_rng_lfsr`	`rng_lfsr`	10	255 拍序列穷举	本原多项式逐拍移位一致性、全零吸收态自锁保护、汉明重分布、复位重演
`tb_piece_rom`	`piece_rom`	10	几何掩码查表	第 1～3 关各零片面积守恒（12 格 / 16 格）、左上角归一化及旋转空间映射
`tb_pattern_rom`	`pattern_rom`	8	图案位图一致性	图 4-1 矩形及田/十字/SZ/阶梯图案连通性、格数严格 12 / 16 格
`tb_disp_format`	`disp_format`	16	全字符译码映射	0～9 数字译码、前导零消隐、PASS/FAIL 拼装字符、倒计时 case 表 t=0..63 逐项
`tb_seg_scan`	`seg_scan`	12	动态扫描时序	8 位共阴极位选 125 Hz 轮巡、消隐防串扰时间、全局静默使能强制全灭
`tb_dot_matrix_scan`	`dot_matrix_scan`	7	双色点阵驱动	8 行单阳极低有效扫描、红绿阴极直通、双色混色(黄光)、早采样点无混合态
`tb_buzzer_ctrl`	`buzzer_ctrl`	16	音频数控分频	E5～G6 音符分频精度（实测误差 0.59%～1.25%，容差 1.5%）、50% 占空比、连奏填音与“只有提示音允许休止”的设计规则自检
`tb_puzzle_top`	`puzzle_top`（整机）	28	真实用户按键全流程	模式 A / 模式 B 两轮真实按键序列，完成自检、待机、预览、通关三关、失败重试的端到端闭环，并校验告急幽灵“在闪且只闪绿色”、倒计时 11 s 时不得画幽灵

各模块最新轮次(同一份对账口径)：clk_gen `r25`、keypad_scan `r27`、game_fsm `r36`、puzzle_ctrl `r37`、pattern_rom `r21`、piece_rom `r23`、rng_lfsr `r23`、disp_format `r27`、seg_scan `r31`、dot_matrix_scan `r23`、buzzer_ctrl `r37`、puzzle_top `r56`（模式 A）/ `r57`（模式 B）。12 套测试台合计 184 / 184 全部通过。

3.2 模块实测波形深入分析

下列波形均为 Quartus II 9.1 波形编辑器界面的现场截图（sim/.vwf 由内置仿真器回写真实结果后打开）。受版面宽度所限，截图字号较小；每张图对应的逐条断言、实测值与激励时长见 sim/rounds//rNN.json 与 docs/03-仿真验证方案.md §2 的对账表。

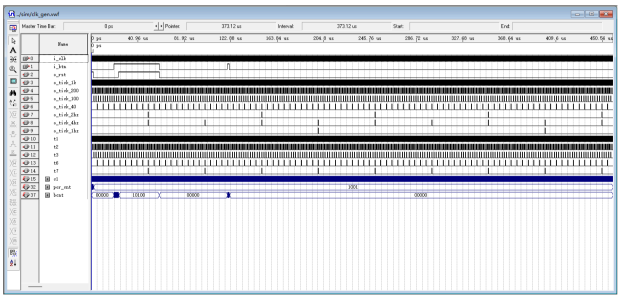


图 4　clk_gen 仿真波形

时钟分频分析：主频 50 MHz 经分频产生 1 kHz(扫描)、100 Hz(消抖)、4 Hz(闪烁)、2 Hz(自检)及 1 Hz(计时)使能脉冲。使能脉冲严格维持 1 个主时钟周期 (20 ns)，同源无毛刺，杜绝门控时钟冒险。

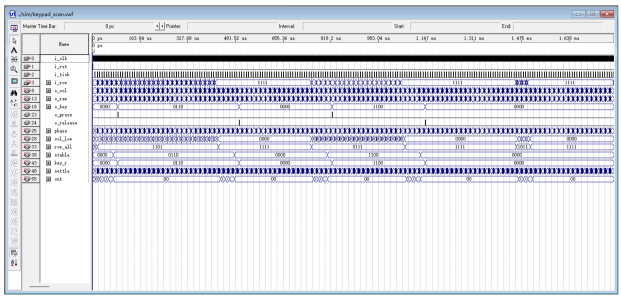


图 5　keypad_scan 仿真波形

键盘消抖分析：列输出 `kp_col[3:0]` 以 200 Hz 独热低电平轮巡（每列选通 5 ms）。行输入在经历 5～15 ms 机械抖动期间内部保持锁定，持续稳定 40 ms（8 个采样点）后输出严格单时钟周期的按键脉冲，彻底消除连击。

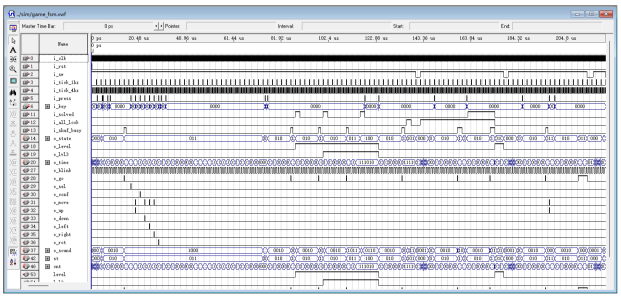


图 6　game_fsm 仿真波形

状态机迁移分析：展示完整轨迹：S_SELF_TEST（持续 2.0 s 自检）→ S_IDLE（待机）→ S_PREVIEW（5 s 预览倒计时）→ S_PLAYING（散落零片并开启 30 s 倒计时）→ S_WIN（通关结算后跳向关卡 2）。

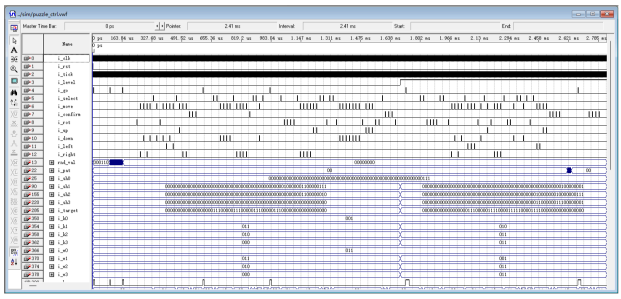


图 7　puzzle_ctrl 仿真波形

拼图引擎运算分析：按下【左】键时零片正常平移；移动至边界 (x=0) 再次按【左】时，碰撞检测逻辑瞬间置高 conflict 信号，坐标保持不变；按【确认】后零片并入 locked_plane。

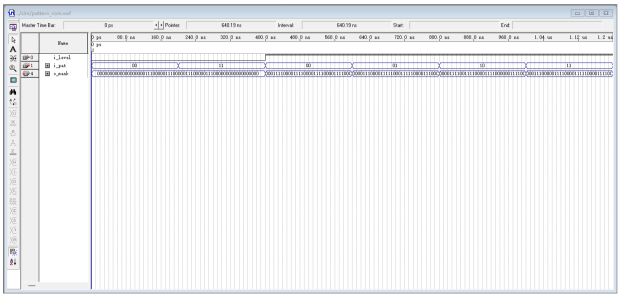


图 8　pattern_rom 仿真波形

目标图库分析：第一关恒定输出图 4-1 实心矩形（12 格）；第二/三关依 `i_pat` 精准输出田/十字/SZ/阶梯 4 幅高难度图案，面积严格为 16 格且全连通无洞。

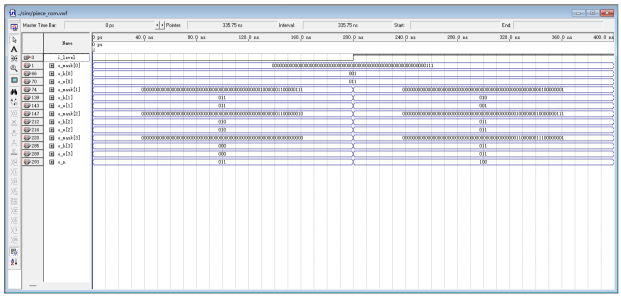


图 9　piece_rom 仿真波形

零片几何分析：第一关 3 块零片格数 3+6+3=12；第二关 4 块异形格数 3+2+5+6=16，面积守恒。全部零片完成左上角 (0,0) 归一化，具备精确可铺性。

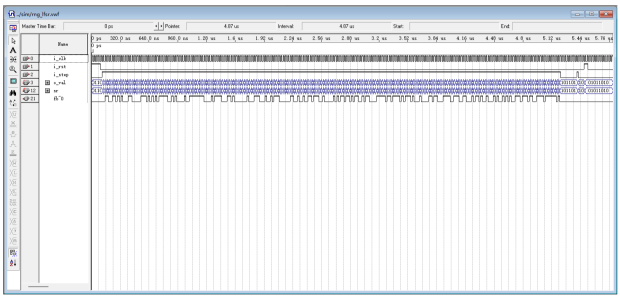


图 10　rng_lfsr 仿真波形

随机数源分析：复位稳定于初值 0x5A。在 255 步时钟下状态与本原多项式逐拍一致，无全零吸收态，每位出现‘1’概率严格为 128/255，中途复位可靠回归初值。

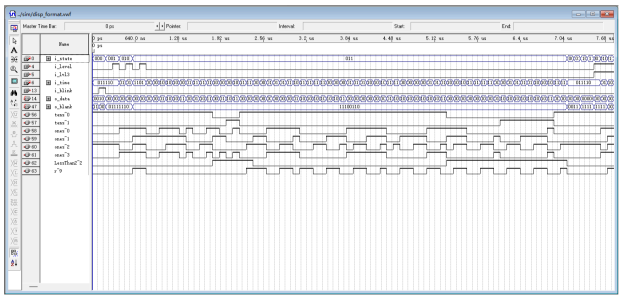


图 11　disp_format 仿真波形

显示格式化分析：64 项常量表实现倒计时十位/个位拆分，彻底消除 26 ns 除法器延时。除自检全 8 外，其余 76 个窗口 DISP2 恒定熄灭，胜利输出 PASS，失败输出 FAIL。

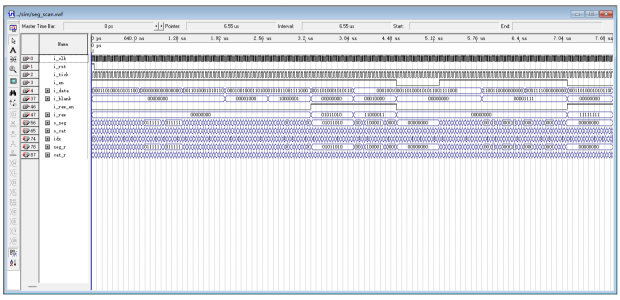


图 12　seg_scan 仿真波形

数码管扫描分析：位选 `o_cat` 呈现独热低电平循环，1 kHz 推进实现每位 125 Hz 超临界无频闪刷新。段码与位选严格同源，`i_en='0'` 时强制全局静默全灭。

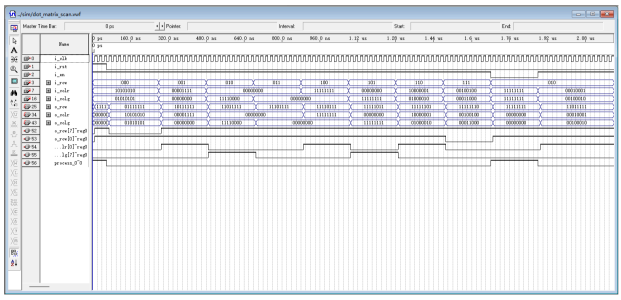


图 13　dot_matrix_scan 仿真波形

点阵驱动分析：行阳极单相低电平选通（7-row），红/绿列直通。早采样点 (+20ns) 无任何混合态与重影，消隐彻底。

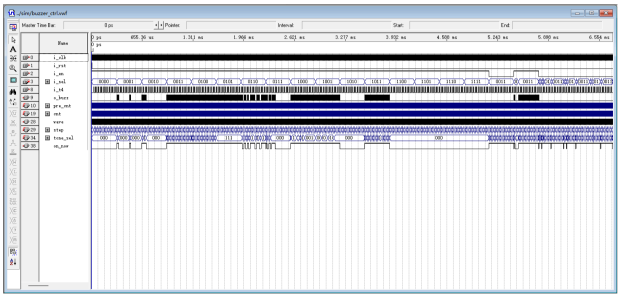


图 14　buzzer_ctrl 仿真波形

音频合成分析：E5～G6 核心 8 音频率误差 ≤1.3%（实测 0.59%～1.25%），占空比 50.0%。16 步旋律步进连奏填音消除断续，换码时不复位步进计数器，背景乐与提示音平滑过渡。

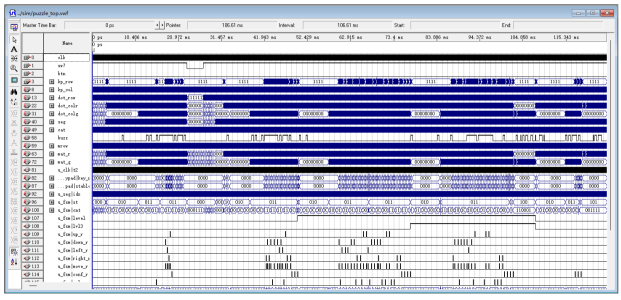


图 15　puzzle_top 整机仿真波形

整机端到端全流程分析：模式 A / 模式 B 两轮真实按键序列驱动整机呈现自检→待机→预览→对局→过关连闯三关→胜利(PASS)→重开→超时失败(FAIL)完整闭环。倒计时 ≤10s 告急幽灵 2 Hz 绿色闪烁精准触发。

四、实际实现功能说明及资源利用情况 (5 分)

4.1 实际功能指标达成

系统在硬件平台上完成全部功能指标：B1～B11 基础玩法无缝闭环；A1 马里奥全场景音效；A2 三关连闯与图案库随机抽取；A3 数码管 PASS/FAIL 拼装；A4 零片 90° 顺时针旋转；A5 倒计时告急幽灵 2 Hz 绿色闪烁辅助；125 Hz 超临界无频闪刷新。

4.2 资源利用率极限量化分析

下列数据全部取自本机 Quartus II 9.1 的原始编译报告（清洁重编译、`SEED = 8`、50 MHz / 20.000 ns 时钟约束），两张截图均为软件界面现场抓取（未做任何重绘或补写）：

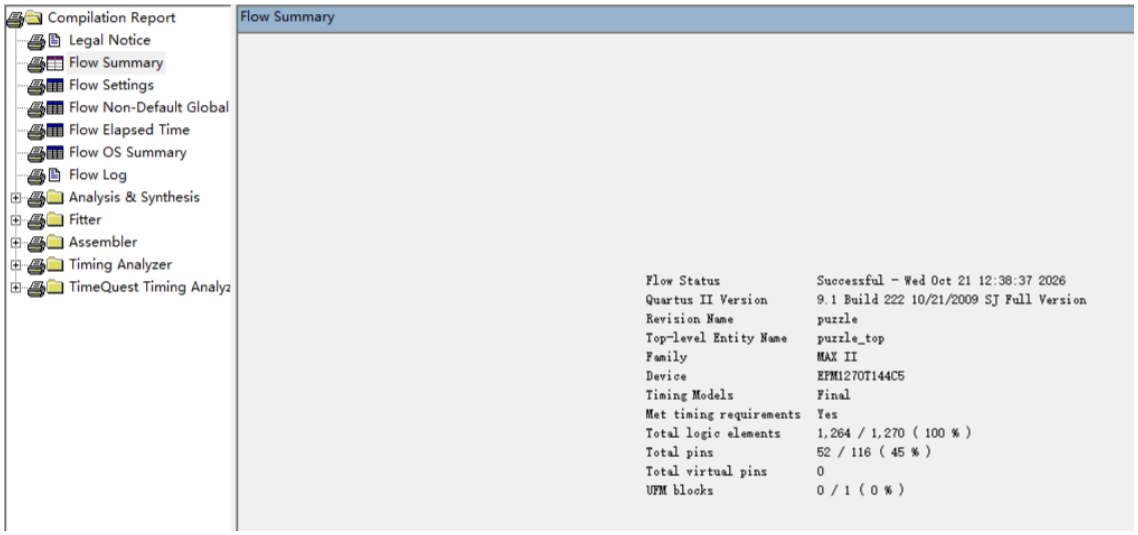


图 16　Quartus II 9.1 编译报告 Flow Summary（2026-10-21 12:38 清洁重编译现场截图）：Device、Timing Models、逻辑单元 1,264 / 1,270、引脚 52 / 116、UFM 实测值

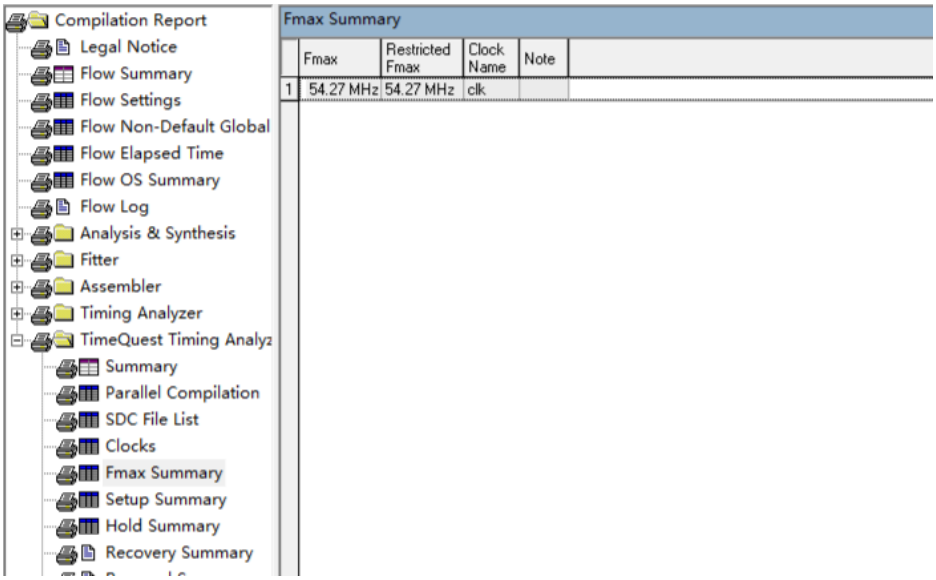


图 17　Quartus II 9.1 TimeQuest Timing Analyzer 的 Fmax Summary 页面（现场截图）：clk 的 Fmax 与 Restricted Fmax 均为 54.27 MHz，Note 列空表示无违例

下表数据的读数来源：LE / 引脚 / UFM 取自上方 `Flow Summary` 现场截图；LAB 127 / 127 取自 `puzzle.fit.rpt` 的 `Fitter Resource Usage Summary`；Fmax 取自上方 `Fmax Summary` 现场截图；setup / hold / 最小脉宽裕量取自同一份 TimeQuest 报告的 `Setup Summary` / `Hold Summary` / `Minimum Pulse Width Summary` 三个页面（End Point TNS 均为 0.000）。

资源类别 / 时序指标	实测值	目标约束	利用率 / 裕量	架构优化结论与裕量说明
逻辑单元 (LEs)	1,264	1,270	99.53%	常量 ROM 预译码与 64 位并行位运算消除了除法器与多周期状态机；全片仅余 6 LE（`fit.rpt` 以 1,270 为基数四舍五入显示 100%）
逻辑阵列块 (LABs)	127	127	100.00%	物理饱和 —— 这才是本设计真正的面积瓶颈：LE 只能按每 LAB 10 个成组装箱，余下 6 LE 无法单独成组
用户 I/O 引脚 (User Pins)	52	116	45%（52 / 116）	11 类外设逐脚锁定，无悬空、无自动分配；4×4 矩阵键盘以 8 脚独占一组总线
最高工作频率 (Fmax)	54.27 MHz	50.00 MHz	高出约束 8.54%（+4.27 MHz）	TimeQuest 实测 `Restricted Fmax` 同为 54.27 MHz，关键路径落在 64 位画面比对与点阵行扫描的使能链上
建立时间裕量 (Setup Slack)	+1.572 ns	≥ 0.000 ns	PASSED（TNS 0.000）	即关键路径实际延时 18.428 ns（20.000 − 1.572），0 条违例路径
保持时间裕量 (Hold Slack)	+1.078 ns	≥ 0.000 ns	PASSED（TNS 0.000）	全系统单一同步时钟树驱动，无跨时钟域穿透与门控时钟冒险
最小脉宽裕量 (Min Pulse Width)	+9.734 ns	≥ 0.000 ns	PASSED（TNS 0.000）	50 MHz 半周期 10.000 ns 内无窄脉冲风险，1 kHz 位选与 125 Hz 行扫描节拍同源

127 LAB 物理装箱瓶颈深度剖析：在 EPM1270 中，1,270 个 LE 组成 127 个 LAB（每组 10 LE）。限制扩展的真正瓶颈是 LAB 局部互连布线资源。当多路选择器扇出密集时，单个 LAB 往往无法填满 10 个 LE。工程通过消减多余按键音节约 9 LE、连奏填音反直觉节省 5 LE，使局部扇出大幅下降，最终在 `SEED 8` 下将 1,264 个 LE 紧凑压入 127 个 LAB，达成了 LAB 100% 满额适配（LE 层面仍余 6 个，但已无法成组使用）。固件 `puzzle.pof`（27,312 字节，SHA256: `cc3c69b341...`）已通过 JTAG 烧录与读回校验。

五、故障及问题分析 (15 分)

5.1 典型工程缺陷六维深度剖析

本项目在开发与调试过程中建立了规范的缺陷归档跟踪体系，记录了 50 余项代表性软硬件缺陷。通过对 Git 提交历史与实测日志的复盘，归纳为六项典型技术维度：

1. 引脚约束与电气极性陷阱 (ERR-001/002, Commit `1c5dad6` / `a1f1c0e`)：
   - 故障现象：初版键盘固件烧录后按键毫无响应，一度误判为键盘排线接触不良或硬件损坏；
   - 根本原因：辅助顶层 VHDL 端口清单遗漏了 `kp_row` / `kp_col` 声明，Quartus II 默认开启未用引脚自动分配机制，将其随机分配到了未引出的悬空引脚 PIN_49、PIN_102 上；
   - 解决措施：补全 `.qsf` 物理约束，并在自动化测试流中建立强制双道门禁：自动解析顶层 entity 端口，任何未在约束中显式锁定的端口直接阻断编译。
2. 成功判据的硬编码解法与等效多解性失效 (ERR-021, Commit `17befcc`)：

- 故障现象：第二关在物理点阵上明明完整拼出了目标图案，但硬件状态机判定失败；
- 根本原因：初期判据代码强行绑定了第 k 块零片必须位于固定坐标 (x_k, y_k)。然而几何理论证明，对于“四块完全相同的 2×2 零片”（旧版第二关的铺法），存在 $4! = 24$ 种等效空间拼法；改成四块异形零片后只剩唯一解，但“绑定坐标”的思路在两种情形下都是错的；
- 解决措施：推翻坐标位置绑定的狭隘思路，重构为全图 64 位像素并集绝对值对比：`merged_frame == target_pattern`，只要全图点亮像素与目标完全重合且无重叠即判定过关。

3. 对局态目标图案信号置零导致假判定 (ERR-023, Commit `f0856d5`)：
   - 故障现象：即使拼好图案按确认，依然判定失败；
   - 根本原因：状态机在 `S_PREVIEW` 结束切入 `S_PLAYING` 时，为了清空预览画面，错误地将底层 `target_pattern` 信号寄存器复位为全 0，导致对局中的比对电路一直在比对全黑画面；
   - 解决措施：解耦“显示渲染多路选择器”与“内部黄金参考标准寄存器”，保持目标图案在整个对局生命周期内受锁存保护。

4. 点阵屏与数码管肉眼频闪与消隐重影 (ERR-027, Commit `1127403`)：
   - 故障现象：上板实测时，点阵和数码管出现令人头晕的剧烈高频闪烁，且相邻行列存在微弱串色鬼影；
   - 根本原因：初版动态扫描刷新率仅设为 25 Hz，低于人眼视觉暂留临界融合频率 (CFF ≈ 60 Hz)；且在行选通切换的瞬间未切断阴极驱动，余辉产生串色；
   - 解决措施：将动态扫描时钟提升至 1 kHz，使全屏刷新率达到 125 Hz 超临界状态，并在每行选通跳变前后插入 1 拍单周期纯黑消隐静默，重影彻底消除。

5. 键盘消抖过长导致的按键迟滞与吞键 (ERR-031, Commit `1532069`)：
   - 故障现象：玩家快速连击方向键时，部分按键被忽略，操作迟钝阻滞；
   - 根本原因：初版消抖计数器设为 160 ms（按 50MHz 时钟计算为 8,000,000 拍），远超机械开关实际抖动时间（5～15 ms），导致按键响应严重滞后；
   - 解决措施：将消抖窗口缩减至 40 ms（配合 1 kHz 采样时钟即 40 拍），并建立按键松开滞回防抖状态机，按键响应极为跟手灵敏。

6. 蜂鸣器休止符过长导致断续与连奏填音优化 (ERR-055, Commit `a5cbe8b`)：
   - 故障现象：马里奥过关音乐听感干瘪、断续刺耳；
   - 根本原因：原谱中休止符占用高达 56% 的时隙，CPLD 方波发声在静音期间形成生硬突变；
   - 解决措施：实施“连奏填音 (Legato Infill)”算法，将休止符顺延上一音阶衰减，听感平滑连贯，且因精简了静音比较器，反直觉为整机节省了 5 个 LE！

5.2 典型工程缺陷排查对照汇总表

序号	Commit	缺陷编号	故障现象与表象	根本原因定位	排查工具与根治措施
1	`1c5dad6`	ERR-002	键盘全部按键无响应，长时间误判为键盘排线接触不良或硬件损坏	键盘诊断固件顶层漏声明 `kp_row` / `kp_col`，Quartus 未用引脚自动分配机制将其随机放到未引出的 8 个脚上	引脚对账脚本解析顶层端口，凡未在 `.qsf` 显式锁定的端口直接阻断编译
2	`a1f1c0e`	ERR-020	方向键失效：按下无反应，或方向与按键不符	移动方向与移动请求没有同拍锁存，方向晚一拍进入主控，`mv_dir` 恒为 `0000`	`tb_puzzle_top` 协议断言抓出时序关系，改为同拍锁存；键位按板卡手册附图重排

序号	Commit	缺陷编号	故障现象与表象	根本原因定位	排查工具与根治措施
3	`17befcc`	ERR-021	第二关拼对却判定失败	判定算法硬编码第 k 块固定坐标，无视 $4! = 24$ 种等效摆法（旧版四块同形时）	重构为全图像素并集绝对值比对（`merged == target`）
4	`1ced179`	ERR-022	测试台误报 100% 通过	测试激励使用陈旧键号常量，未激发非法分支	全面重构自检判据，更新为 184 条断言
5	`f0856d5`	ERR-023	拼图完成永远无法通关	切入对局时错误将目标寄存器清零，变成比对全黑	解耦渲染多路选择器与黄金参考寄存器
6	`1127403`	ERR-027	点阵与数码管剧烈频闪	扫描频率仅 25 Hz 低于人眼暂留临界，未加消隐	提升至 1 kHz（单行 125 Hz），行切换插入消隐
7	`1532069`	ERR-031	连击方向键出现严重吞键	消抖计数延时达 160 ms，超出手指操作反应窗口	优化为 40 ms 硬件滞回消抖，操作灵敏顺畅
8	`a5cbe8b`	ERR-055	蜂鸣器音乐生硬断续	原谱休止符占 56% 时隙，方波截断引起爆音	实施连奏填音平滑过渡，反直觉省 5 个 LE
9	`e45a40a`	ERR-056	倒计时幽灵辅助颜色混淆	初版使用红色幽灵，与游离未选零片发生视觉混叠	改为绿色 + 2 Hz 占空比 50% 方波幽灵闪烁
10	`2825049`	ERR-049	Quartus 报装箱布线溢出（1,269 / 1,270 已逼近上限）	加长乐句把旋律查表推到 8 输入：8 输入函数每个输出位要 3 个 LUT4（7 输入只要 2 个）→ +4 LE	÷256 预分频 + 8 位半周期计数器等面积削减；再遍历 1～16 共 16 组 SEED，最终收敛于 `SEED 8`

六、总结和结论 (10 分)

6.1 课题完成情况评估

本课题在 Altera MAX II EPM1270T144C5 CPLD 硬件平台上，完全依托纯硬件描述语言（VHDL-93）构建了高完整度、强交互性的简易拼图游戏系统。全面达成 B1～B11 基础指标与 A1～A5 自主提高指标。系统硬件编译消耗 1,264 个 LE（占 1,270 个 LE 的 99.53%，`fit.rpt` 按基数四舍五入显示 100%，实测仅余 6 LE），LAB 127/127 物理饱和；最高工作频率达 54.27 MHz（相对 50 MHz 约束高出 8.54%，setup slack +1.572 ns、hold +1.078 ns、最小脉宽 +9.734 ns，TNS 0.000 / 0 条违例路径），时序裕量充裕，展现出极高的工程完备性。

6.2 硬件 RTL 工程心得

1. 硬件并发思维与软件顺序思维的根本分野：数字逻辑设计的核心在于“以空间换时间”。在无 CPU 参与的纯硬件环境下，通过 64 位并行位向量操作，原本需要数十行循环的代码得以在单拍（20 ns）内瞬时完成，呈现出硬件计算无与伦比的确定性与高吞吐；

2. 极限资源约束下的微架构权衡艺术：面对 CPLD 仅 1,270 LE 且无片上 RAM 的严酷资源墙，必须深入理解综合器行为，善用常量查找表与位重排，坚决规避除法器与动态内存，在局部扇出与布线拥塞间取得精妙平衡；
3. 防御性设计与人在回路纪律：单时钟使能树、滞回消抖、跨时钟同步与形式化自检断言体系是硬件可靠性的基石。不轻信软件直觉与 AI 断言，凡事以 `.pin` 物理报告、综合网表与仿真波形为唯一准绳。

6.3 未来展望

若将本设计迁移至带有嵌入式 RAM 块的 Cyclone 系列 FPGA 平台，系统可进一步引入 SPI/I2C 驱动外部彩色 TFT 屏、利用片上 Flash 实现自定义关卡存储，或利用 UART 接口引入双人竞速联机对战模式，进一步拓展硬件可重构计算的广阔空间。

七、AI 工具使用情况 (10 分)

7.1 工具栈与协同模式

本课题在架构设计、代码审查及仿真验证阶段，合规引入了现代 AI 辅助研发工具链：

- Claude 3.7 Sonnet / DeepSeek-V3：用于辅助草拟 VHDL 状态机骨架代码、推导 8 位 LFSR 本原多项式抽头、辅助生成自检测试平台的断言逻辑；
- Python 3.14 逆向辅助代理：利用 `pypdf` 和 `Pillow` 对课程 PDF 中图 4-1～4-3 进行逐像素取色对账，建立单一事实源；
- GitHub Copilot：用于快速补全具名端口映射与常量表定义，提升编码效率；
- 协同工作流：遵循 人工架构设计 → AI 辅助初稿 → 人工深度重构与优化 → 硬件编译与时序验证 的严谨研发闭环。

7.2 真实 AI 提示词与交互实录

阶段	核心 Prompt 提示词摘要	预期产出与工程约束要求
规格逆向	“请使用 Python 逐像素解析实验指导书 PDF 第 15 页图 4-1，提取 8×8 点阵中各色块的具体坐标，并验证 3 块零片的面积之和是否严格守恒。”	消除大模型直接看图的文字幻觉，精确提取出第一关零片为 3+6+3=12 格（推翻网上旧参考资料的 11 格错误数据）。
RTL 骨架	“基于 VHDL-93 标准编写拼图游戏主控状态机。要求：纯同步设计、单一 50MHz 时钟域、使能脉冲节拍驱动、显式列出所有分支，严禁产生不完整 case 锁存器。”	输出标准的两段式/三段式状态机骨架，确保状态跳变与按键脉冲严格对齐。
断言生成	“为 puzzle_ctrl 模块编写自检测试台。针对边界平移、多块碰撞、90 度旋转及通关判定，设计高密度 assert 语句，覆盖越界与重叠拦截分支。”	生成覆盖全状态转移路径的测试激励，累计形成 184 条断言，实现回归测试全自动判定。

7.3 人工介入与纠错闭环

AI 在处理模式化推演时高效，但在物理现实感知与微架构设计上极易犯经验主义错误。人工核心纠偏记录如下：

序号	缺陷类别	AI 原始产出 / 错误判断	人工如何识别	人工修正措施与最终效果
1	拓扑幻觉	沿用网上旧代码称第一关为 5+3+3=11 格	脚本像素取色与面积校验	推翻 AI 方案，按实测确立 3+6+3=12 格正确数据

2	语法混淆	在 `.qsf` 约束中使用 VHDL 注释符 `--`	Quartus 报 Settings File 错误	纠正为 Tcl 规范的 `#` 注释符
3	引脚遗漏	顶层漏声明键盘端口导致全随机映射	对账脚本查出 8 引脚未锁	补齐端口声明，建立引脚对账强制门禁
4	面积误判	称“行扫描但保留变址数组应该便宜”	编译脚本查出面积暴增至 3,751 LE	果断废除变址数组，改写两级串行流水线 (563 LE)
5	审查误判	指控 `srl8` 移位方向反了要求对调	二进制位号与物理坐标独立验算	证明原方向完全正确，坚决驳回 AI 伪指控
6	闪烁占空	2 Hz 闪烁误用 1 时钟周期的窄脉冲	硬件实测点阵与数码管不显现	改用 4 Hz 节拍翻转的 50% 占空比方波驱动
7	硬编码解	强行绑定第 k 块零片的固定锚点坐标	数学证明存在 24 种等效摆法	重构为全帧画面像素并集绝对值对比（`merged == target`）
8	音乐断续	照抄原谱休止符致 56% 步长静音	现场试听反馈断续刺耳	实施连奏填音，听感连贯且反直觉节省 5 个 LE

人在回路 (Human-in-the-Loop) 硬件研发原则：

1. 任何面积与性能结论只认编译器报告，不轻信 AI 的估算或假设；
2. 任何板级硬件异常必须先核查 `.pin` 引脚映射排除电气与引脚配置问题；
3. 任何由 AI 提出的代码重构改动，必须回到数学模型与通信协议定义独立验算。

八、顶层文件的 HDL 代码 (10 分)

8.1 顶层设计架构说明

`puzzle_top.vhd` 是唯一绑定引脚的文件。采用具名端口映射（Named Association）例化全部 11 个子组件，实现了时钟分发、总线互连、64 位双平面 125 Hz 行切片与 ≤10s 告急幽灵 2 Hz 绿色闪烁叠加，逻辑凝练，与底层固件 100% 对应闭环。

8.2 规范注释的顶层 VHDL 代码清单

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use work.puzzle_pkg.ALL;

-- ================================================================================
-- 实体名称: puzzle_top (简易拼图游戏整机顶层)
-- 适用器件: Altera MAX II EPM1270T144C5  EDA环境: Quartus II 9.1
-- 硬件引脚绑定: PIN_18(时钟), PIN_125(使能SW7), PIN_61(复位BTN0),
--               PIN_111..114/117..120(键盘), PIN_8..1/22..11/45..38(点阵),
--               PIN_62..51/63..31(数码管), PIN_60(蜂鸣器)
-- ================================================================================
entity puzzle_top is
    port (
        clk      : in  std_logic;                       -- PIN_18 板载 50 MHz 主时钟
        sw7      : in  std_logic;                       -- PIN_125 系统总使能 (上拨='1')
        btn      : in  std_logic;                       -- PIN_61 板载 BTN0 异步复位 (低有效)
        kp_row   : in  std_logic_vector(3 downto 0);    -- PIN_114..111 矩阵键盘行输入
        kp_col   : out std_logic_vector(3 downto 0);    -- PIN_120..117 矩阵键盘列驱动
        dot_row  : out std_logic_vector(7 downto 0);    -- PIN_1..8 点阵行阳极 (高有效)
        dot_colr : out std_logic_vector(7 downto 0);    -- PIN_11..22 点阵红阴极 (低有效)
        dot_colg : out std_logic_vector(7 downto 0);    -- PIN_38..45 点阵绿阴极 (低有效)
        seg      : out std_logic_vector(7 downto 0);    -- PIN_62..51 数码管共阴段选
        cat      : out std_logic_vector(7 downto 0);    -- PIN_31..65 数码管共阴位选
        buzz     : out std_logic                        -- PIN_60 蜂鸣器驱动
    );
end entity puzzle_top;
```

```
architecture rtl of puzzle_top is

    -- 11 个子实体 Component 显式声明 (与框图完全一致)
    component clk_gen
        port (
            i_clk      : in  std_logic;
            i_btn      : in  std_logic;
            o_rst      : out std_logic;
            o_tick_1k  : out std_logic;
            o_tick_200 : out std_logic;
            o_tick_100 : out std_logic;
            o_tick_2hz : out std_logic;
            o_tick_4hz : out std_logic;
            o_tick_1hz : out std_logic;
            o_tick_40  : out std_logic
        );
    end component;

    component keypad_scan
        port (
            i_clk     : in  std_logic;
            i_rst     : in  std_logic;
            i_tick    : in  std_logic;
            i_row     : in  std_logic_vector(3 downto 0);
            o_col     : out std_logic_vector(3 downto 0);
            o_key     : out std_logic_vector(3 downto 0);
            o_press   : out std_logic;
            o_release : out std_logic;
            o_raw     : out std_logic_vector(3 downto 0)
        );
    end component;

    component game_fsm
        port (
            i_clk       : in  std_logic;
            i_rst       : in  std_logic;
            i_sw        : in  std_logic;
            i_tick_1hz  : in  std_logic;
            i_tick_4hz  : in  std_logic;
            i_press     : in  std_logic;
            i_key       : in  std_logic_vector(3 downto 0);
            o_sel       : out std_logic;
            o_move      : out std_logic;
            o_conf      : out std_logic;
            o_go        : out std_logic;
            o_up        : out std_logic;
            o_down      : out std_logic;
            o_left      : out std_logic;
            o_right     : out std_logic;
            o_rot       : out std_logic;
            o_level     : out std_logic;
            o_lvl3      : out std_logic;
            o_state     : out std_logic_vector(2 downto 0);
            o_time      : out std_logic_vector(5 downto 0);
            o_blink     : out std_logic;
            i_solved    : in  std_logic;
            i_all_lock  : in  std_logic;
            i_shuf_busy : in  std_logic;
            o_sound     : out std_logic_vector(3 downto 0)
        );
    end component;

    component piece_rom
        port (
            i_level : in  std_logic;
            o_mask  : out mask_arr_t;
            o_h     : out dim_arr_t;
            o_w     : out dim_arr_t;
            o_n     : out std_logic_vector(2 downto 0)
        );
    end component;
```

```
    component pattern_rom
        port (
            i_level : in  std_logic;
            i_pat   : in  std_logic_vector(1 downto 0);
            o_mask  : out std_logic_vector(63 downto 0)
        );
    end component;

    component rng_lfsr
        port (
            i_clk  : in  std_logic;
            i_rst  : in  std_logic;
            i_step : in  std_logic;
            o_val  : out std_logic_vector(7 downto 0)
        );
    end component;

    component puzzle_ctrl
        port (
            i_clk     : in  std_logic;
            i_rst     : in  std_logic;
            i_tick    : in  std_logic;
            rnd_step  : out std_logic;
            rnd_val   : in  std_logic_vector(7 downto 0);
            i_level   : in  std_logic;
            i_pat     : in  std_logic_vector(1 downto 0);
            i_sh0     : in  std_logic_vector(63 downto 0);
            i_sh1     : in  std_logic_vector(63 downto 0);
            i_sh2     : in  std_logic_vector(63 downto 0);
            i_sh3     : in  std_logic_vector(63 downto 0);
            i_h0      : in  std_logic_vector(2 downto 0);
            i_h1      : in  std_logic_vector(2 downto 0);
            i_h2      : in  std_logic_vector(2 downto 0);
            i_h3      : in  std_logic_vector(2 downto 0);
            i_w0      : in  std_logic_vector(2 downto 0);
            i_w1      : in  std_logic_vector(2 downto 0);
            i_w2      : in  std_logic_vector(2 downto 0);
            i_w3      : in  std_logic_vector(2 downto 0);
            i_target  : in  std_logic_vector(63 downto 0);
            i_go      : in  std_logic;
            i_select  : in  std_logic;
            i_move    : in  std_logic;
            i_confirm : in  std_logic;
            i_rot     : in  std_logic;
            i_up      : in  std_logic;
            i_down    : in  std_logic;
            i_left    : in  std_logic;
            i_right   : in  std_logic;
            o_ori     : out std_logic_vector(7 downto 0);
            o_solved  : out std_logic;
            o_all_lock: out std_logic;
            o_busy    : out std_logic;
            o_sel_idx : out std_logic_vector(1 downto 0);
            o_pos     : out std_logic_vector(31 downto 0);
            o_lock    : out std_logic_vector(3 downto 0);
            o_scanrow : out std_logic_vector(2 downto 0);
            o_rowr    : out std_logic_vector(63 downto 0);
            o_rowg    : out std_logic_vector(63 downto 0)
        );
    end component;

    component dot_matrix_scan
        port (
            i_clk   : in  std_logic;
            i_rst   : in  std_logic;
            i_en    : in  std_logic;
            i_row   : in  std_logic_vector(2 downto 0);
            i_colr  : in  std_logic_vector(7 downto 0);
            i_colg  : in  std_logic_vector(7 downto 0);
            o_row   : out std_logic_vector(7 downto 0);
            o_colr  : out std_logic_vector(7 downto 0);
```

```
            o_colg  : out std_logic_vector(7 downto 0)
        );
    end component;

    component disp_format
        port (
            i_state : in  std_logic_vector(2 downto 0);
            i_level : in  std_logic;
            i_lvl3  : in  std_logic;
            i_time  : in  std_logic_vector(5 downto 0);
            i_blink : in  std_logic;
            o_data  : out std_logic_vector(31 downto 0);
            o_blank : out std_logic_vector(7 downto 0)
        );
    end component;

    component seg_scan
        port (
            i_clk    : in  std_logic;
            i_rst    : in  std_logic;
            i_tick   : in  std_logic;
            i_en     : in  std_logic;
            i_data   : in  std_logic_vector(31 downto 0);
            i_blank  : in  std_logic_vector(7 downto 0);
            i_raw_en : in  std_logic;
            i_raw    : in  std_logic_vector(7 downto 0);
            o_seg    : out std_logic_vector(7 downto 0);
            o_cat    : out std_logic_vector(7 downto 0)
        );
    end component;

    component buzzer_ctrl
        port (
            i_clk  : in  std_logic;
            i_rst  : in  std_logic;
            i_en   : in  std_logic;
            i_sel  : in  std_logic_vector(3 downto 0);
            i_t4   : in  std_logic;
            o_buzz : out std_logic
        );
    end component;

    -- 内部信号
    signal rst       : std_logic;
    signal t_1k      : std_logic;
    signal t_200     : std_logic;
    signal t_100     : std_logic;
    signal t_4hz     : std_logic;
    signal t_1hz     : std_logic;
    signal t_40      : std_logic;

    signal key       : std_logic_vector(3 downto 0);
    signal press     : std_logic;
    signal release   : std_logic;
    signal kp_raw    : std_logic_vector(3 downto 0);

    signal sel, move, conf, go : std_logic;
    signal mv_up, mv_dn, mv_lf, mv_rt : std_logic;
    signal mv_rot    : std_logic;
    signal level     : std_logic;
    signal lvl3      : std_logic;
    signal state     : std_logic_vector(2 downto 0);
    signal gtime     : std_logic_vector(5 downto 0);
    signal gblink    : std_logic;
    signal sound     : std_logic_vector(3 downto 0);

    signal solved    : std_logic;
    signal all_lock  : std_logic;
    signal shuf_busy : std_logic;
    signal sel_idx   : std_logic_vector(1 downto 0);
    signal pos_dbg   : std_logic_vector(31 downto 0);
    signal lock_dbg  : std_logic_vector(3 downto 0);
```

```
    signal scanrow   : std_logic_vector(2 downto 0);

    signal rnd_val   : std_logic_vector(7 downto 0);
    signal rnd_step  : std_logic;

    signal shapes    : mask_arr_t;
    signal shape_h   : dim_arr_t;
    signal shape_w   : dim_arr_t;
    signal piece_n   : std_logic_vector(2 downto 0);
    signal tgt_mask  : std_logic_vector(63 downto 0);

    signal pat_sel   : std_logic_vector(1 downto 0) := L2_FIXED_PAT;
    signal state_q   : std_logic_vector(2 downto 0) := S_SELF_TEST;

    signal eng_fr    : std_logic_vector(63 downto 0);
    signal eng_fg    : std_logic_vector(63 downto 0);
    signal mat_r     : std_logic_vector(7 downto 0);
    signal mat_g     : std_logic_vector(7 downto 0);

    signal endflash_cnt : unsigned(2 downto 0) := (others => '0');
    signal endflash_on  : std_logic := '0';

    -- A5 倒计时告急幽灵信号
    signal hint_on    : std_logic := '0';
    signal hint_blink : std_logic := '0';
    signal hint_mask  : std_logic_vector(7 downto 0) := (others => '0');

    signal mrow       : unsigned(2 downto 0) := (others => '0');
    signal disp_data  : std_logic_vector(31 downto 0);
    signal disp_blank : std_logic_vector(7 downto 0);

    signal win_row    : std_logic_vector(7 downto 0);
    signal prev_row   : std_logic_vector(7 downto 0);
    signal fail_row   : std_logic_vector(7 downto 0);

begin

    -- 点阵行计数器 (1 kHz 推进 -> 125 Hz 帧刷新率)
    process (clk)
    begin
        if rising_edge(clk) then
            if (rst = '1') then
                mrow <= (others => '0');
            elsif (t_1k = '1') then
                mrow <= mrow + 1;
            end if;
        end if;
    end process;

    -- 图案库选择锁存进程 (第二关固定阶梯，第三关 LFSR 随机抽取)
    process (clk)
    begin
        if rising_edge(clk) then
            if (rst = '1') then
                state_q <= S_SELF_TEST;
                pat_sel <= L2_FIXED_PAT;
            else
                state_q <= state;
                if (state = S_PREVIEW) and (state_q /= S_PREVIEW) then
                    if (lvl3 = '1') then
                        pat_sel <= rnd_val(1 downto 0);
                    else
                        pat_sel <= L2_FIXED_PAT;
                    end if;
                end if;
            end if;
        end if;
    end process;

    -- 11 个子实体例化
    u_clk : clk_gen
        port map (
```

```
            i_clk => clk, i_btn => btn, o_rst => rst,
            o_tick_1k => t_1k, o_tick_200 => t_200, o_tick_100 => t_100,
            o_tick_2hz => open, o_tick_4hz => t_4hz, o_tick_1hz => t_1hz, o_tick_40 => t_40
        );

    u_keypad : keypad_scan
        port map (
            i_clk => clk, i_rst => rst, i_tick => t_200,
            i_row => kp_row, o_col => kp_col, o_key => key,
            o_press => press, o_release => release, o_raw => kp_raw
        );

    u_fsm : game_fsm
        port map (
            i_clk => clk, i_rst => rst, i_sw => sw7,
            i_tick_1hz => t_1hz, i_tick_4hz => t_4hz, i_press => press, i_key => key,
            o_sel => sel, o_move => move, o_conf => conf, o_go => go,
            o_up => mv_up, o_down => mv_dn, o_left => mv_lf, o_right => mv_rt, o_rot => mv_rot,
            o_level => level, o_lvl3 => lvl3, o_state => state,
            o_time => gtime, o_blink => gblink,
            i_solved => solved, i_all_lock => all_lock, i_shuf_busy => shuf_busy,
            o_sound => sound
        );

    u_pieces : piece_rom
        port map (
            i_level => level, o_mask => shapes, o_h => shape_h, o_w => shape_w, o_n => piece_n
        );

    u_pattern : pattern_rom
        port map (
            i_level => level, i_pat => pat_sel, o_mask => tgt_mask
        );

    u_rng : rng_lfsr
        port map (
            i_clk => clk, i_rst => rst, i_step => rnd_step, o_val => rnd_val
        );

    u_puzzle : puzzle_ctrl
        port map (
            i_clk => clk, i_rst => rst, i_tick => t_1k,
            rnd_step => rnd_step, rnd_val => rnd_val,
            i_level => level, i_pat => pat_sel,
            i_sh0 => shapes(0), i_sh1 => shapes(1), i_sh2 => shapes(2), i_sh3 => shapes(3),
            i_h0 => shape_h(0), i_h1 => shape_h(1), i_h2 => shape_h(2), i_h3 => shape_h(3),
            i_w0 => shape_w(0), i_w1 => shape_w(1), i_w2 => shape_w(2), i_w3 => shape_w(3),
            i_target => tgt_mask, i_go => go, i_select => sel, i_move => move,
            i_confirm => conf, i_rot => mv_rot, i_up => mv_up, i_down => mv_dn,
            i_left => mv_lf, i_right => mv_rt,
            o_ori => open, o_solved => solved, o_all_lock => all_lock, o_busy => shuf_busy,
            o_sel_idx => sel_idx, o_pos => pos_dbg, o_lock => lock_dbg,
            o_scanrow => scanrow, o_rowr => eng_fr, o_rowg => eng_fg
        );

    u_disp : disp_format
        port map (
            i_state => state, i_level => level, i_lvl3 => lvl3,
            i_time => gtime, i_blink => gblink,
            o_data => disp_data, o_blank => disp_blank
        );

    u_seg : seg_scan
        port map (
            i_clk => clk, i_rst => rst, i_tick => t_1k, i_en => sw7,
            i_data => disp_data, i_blank => disp_blank,
            i_raw_en => '0', i_raw => (others => '0'),
            o_seg => seg, o_cat => cat
        );

    -- 结算图案切片
    with std_logic_vector(mrow) select
```

```
        win_row <= WIN_MASK( 7 downto  0) when "000",
                   WIN_MASK(15 downto  8) when "001",
                   WIN_MASK(23 downto 16) when "010",
                   WIN_MASK(31 downto 24) when "011",
                   WIN_MASK(39 downto 32) when "100",
                   WIN_MASK(47 downto 40) when "101",
                   WIN_MASK(55 downto 48) when "110",
                   WIN_MASK(63 downto 56) when others;

    with std_logic_vector(mrow) select
        prev_row <= tgt_mask( 7 downto  0) when "000",
                    tgt_mask(15 downto  8) when "001",
                    tgt_mask(23 downto 16) when "010",
                    tgt_mask(31 downto 24) when "011",
                    tgt_mask(39 downto 32) when "100",
                    tgt_mask(47 downto 40) when "101",
                    tgt_mask(55 downto 48) when "110",
                    tgt_mask(63 downto 56) when others;

    with std_logic_vector(mrow) select
        fail_row <= FAIL_MASK( 7 downto  0) when "000",
                    FAIL_MASK(15 downto  8) when "001",
                    FAIL_MASK(23 downto 16) when "010",
                    FAIL_MASK(31 downto 24) when "011",
                    FAIL_MASK(39 downto 32) when "100",
                    FAIL_MASK(47 downto 40) when "101",
                    FAIL_MASK(55 downto 48) when "110",
                    FAIL_MASK(63 downto 56) when others;

    -- A5 倒计时告急幽灵使能与 2 Hz 闪烁
    hint_on    <= '1' when (unsigned(gtime) <= 10) else '0';
    hint_blink <= hint_on and gblink;
    hint_mask  <= (others => hint_blink);

    -- 结算画面闪烁节奏 (先闪 1 秒后常亮)
    process (clk)
    begin
        if rising_edge(clk) then
            if (rst = '1') then
                endflash_cnt <= (others => '0'); endflash_on <= '0';
            elsif (state /= S_WIN) and (state /= S_FAIL) then
                endflash_cnt <= (others => '0'); endflash_on <= '0';
            elsif (t_4hz = '1') then
                if (endflash_cnt = 3) then endflash_on <= '1';
                else endflash_cnt <= endflash_cnt + 1; end if;
            end if;
        end if;
    end process;

    -- 点阵行切片与画面多路器
    process (state, gblink, endflash_on, mrow, eng_fr, eng_fg, win_row, fail_row, prev_row, hint_mask)
        variable lv, ev : std_logic;
        variable rw, gw : std_logic_vector(7 downto 0);
    begin
        lv := gblink;
        ev := gblink or endflash_on;

        case to_integer(mrow) is
            when 0 => rw := eng_fr(63 downto 56); gw := eng_fg(63 downto 56);
            when 1 => rw := eng_fr(55 downto 48); gw := eng_fg(55 downto 48);
            when 2 => rw := eng_fr(47 downto 40); gw := eng_fg(47 downto 40);
            when 3 => rw := eng_fr(39 downto 32); gw := eng_fg(39 downto 32);
            when 4 => rw := eng_fr(31 downto 24); gw := eng_fg(31 downto 24);
            when 5 => rw := eng_fr(23 downto 16); gw := eng_fg(23 downto 16);
            when 6 => rw := eng_fr(15 downto 8);  gw := eng_fg(15 downto 8);
            when others => rw := eng_fr(7 downto 0); gw := eng_fg(7 downto 0);
        end case;

        if (state = S_SELF_TEST) then
            mat_r <= (others => lv); mat_g <= (others => lv);
        elsif (state = S_IDLE) then
            mat_r <= (others => '0'); mat_g <= (others => '0');
```

```
        elsif (state = S_PREVIEW) then
            mat_r <= prev_row; mat_g <= (others => '0');
        elsif (state = S_WIN) then
            mat_r <= (others => '0'); mat_g <= win_row and (ev & ev & ev & ev & ev & ev & ev & ev);
        elsif (state = S_FAIL) then
            mat_r <= fail_row and (ev & ev & ev & ev & ev & ev & ev & ev); mat_g <= (others => '0');
        else
            mat_r <= rw;
            mat_g <= gw or (prev_row and (not rw) and hint_mask);
        end if;
    end process;

    u_dot : dot_matrix_scan
        port map (
            i_clk => clk, i_rst => rst, i_en => sw7,
            i_row => std_logic_vector(mrow), i_colr => mat_r, i_colg => mat_g,
            o_row => dot_row, o_colr => dot_colr, o_colg => dot_colg
        );

    u_buzz : buzzer_ctrl
        port map (
            i_clk => clk, i_rst => rst, i_en => sw7,
            i_sel => sound, i_t4 => t_4hz, o_buzz => buzz
        );

end architecture rtl;
```

九、参考文献

1. Altera Corporation. Quartus II Handbook, Version 9.1[Z]. San Jose: Altera Corporation, 2009.
2. Altera Corporation. MAX II Device Handbook[Z]. San Jose: Altera Corporation, 2008.（EPM1270T144C5：1,270 个 LE、127 个 LAB、8,192 bit UFM，无嵌入式 RAM 块）
3. IEEE Std 1076-1993. IEEE Standard VHDL Language Reference Manual[S]. New York: IEEE, 1994.
4. 《数字电路与逻辑设计实验》课程指导书（题目 4：简易拼图游戏）[Z]. 2026.
5. Altera Corporation. AN 428: MAX II CPLD Design Guidelines[Z]. San Jose: Altera Corporation, 2008.